



DEPARTMENT OF
COMPUTER SCIENCE



RECURSION II

DR JEREMIAH O. BANDELE

PhD Electrical and Electronic Engineering

University of Nottingham



LEARNING OBJECTIVES

At the end of the lesson, you should be able to:

- Compare normal and recursive functions.
- Interpret recursive functions.
- Describe various types of recursive functions.



FACTORIAL

A factorial, written as $n!$, is defined as

$$n! = 1 * 2 * 3 * \dots * (n-2) * (n-1) * n$$

Can we see an example?



FACTORIAL: NORMAL FUNCTION

This simple iterative function mirrors the definition of factorial.

```
int fact(int n) {  
    int i;  
    int result;  
  
    result = 1;  
    for (i = 1; i <= n; i++) {  
        result = result * i;  
    }  
    return result;  
}
```



FACTORIAL: RECURSIVE FUNCTION

This function is recursive, because it contains a call to itself.

```
5  int fact(int n) {  
6      if (n == 1) {  
7          return 1;  
8      }  
9      return n * fact(n - 1);  
10 }
```



FACTORIAL: ITERATIVE VS RECURSIVE FUNCTION

Recursion simplifies the function by making the computer do more work while we do less work.

Iterative Function	Recursive Function
Has two local variables	Has none
Has three statements	Has one
Saves the solution in an intermediate variable before it can be returned	Calculates and returns its result as a single expression



WRITING RECURSIVE FUNCTIONS

To successfully use recursion to solve a problem, we must be able to break the problem down into subproblems, that are similar in form to the original problem.

Example: Count the number of occurrences of the number 42 in an array of n integers.

First, write the header

```
int count42Ints(int arr[], int n);
```



WRITING RECURSIVE FUNCTIONS: SUBPROBLEMS

Breaking the problem at hand into subproblems involves;

- Counting how often 42 appears in the first $n-1$ elements of the array.
- Counting how often 42 appears in the n -th element of the array.
- Adding the two sums and return the result.



WRITING RECURSIVE FUNCTIONS: SUBPROBLEMS

The following code handles the subproblems

```
7   if (arr[n-1] != 42) {  
8       return count42Ints(arr, n-1);  
9   }  
10  return 1 + count42Ints(arr, n-1);
```



WRITING RECURSIVE FUNCTIONS: SOLUTION 1

We need to now indicate a base case; which is when $n = 0$

```
12  int count42Ints(int arr[], int n) {  
13      if (n==0) return 0;  
14      if (arr[n-1] != 42) {  
15          return count42Ints(arr, n-1);  
16      }  
17      return 1 + count42Ints(arr, n-1);  
18  }
```



WRITING RECURSIVE FUNCTIONS

We can use another recursive approach to solve the same problem.

It involves breaking the array into two pieces of equal size, counting the number of 42s in each half and then, adding the two sums.

The header will be different.

```
int count42Ints(int arr[], int low, int high);
```



WRITING RECURSIVE FUNCTIONS: SUBPROBLEM

The number of 42s in each half of the array is then counted and added together.

```
count42Ints(arr, low, (low + high) / 2) +  
    count42Ints(arr, (low + high) / 2 + 1, high);
```



WRITING RECURSIVE FUNCTIONS: SOLUTION 2

We need to now indicate a base case for the solution to be complete.

```
25  int count42Ints(int arr[], int low, int high) {
26      if ((low > high) || (low == high && arr[low] != 42)) {
27          return 0 ;
28      }
29      if (low == high && arr[low] == 42) {
30          return 1;
31      }
32      return count42Ints(arr, low, (low + high)/2) +
33             count42Ints(arr, 1 + (low + high)/2, high);
34  }
```



RECURSIVE PROGRAMS: GENERAL APPROACH

- Write the function header which shows what the function will do and how it will be called.
- Decompose the problem into subproblems.
- Write recursive calls to solve those subproblems whose form is similar to that of the original problem.



RECURSIVE PROGRAMS: GENERAL APPROACH

- Write code to combine, augment, or modify the results of the recursive call(s) if necessary to construct the desired return value or create the desired side effects.
- Write base case(s) to handle any situations that are not handled properly by the recursive portion of the program.



RECURSION: DIRECT

Contains an explicit call to itself

```
37  int func(int x) {  
38      if (x <= 0) {  
39          return x;  
40      }  
41      return foo(x - 1);  
42  }
```



RECURSION: INDIRECT

A function contains a call to another function which eventually calls the function.

```
44  int func1(int x) {  
45      if (x <= 0) {  
46          return x;  
47      }  
48      return func2(x);  
49  }  
50  
51  int func2(int y) {  
52      return func1(y - 1);  
53  }
```



RECURSION: TAIL

A recursive function without pending operations to be performed on return from a recursive call.

```
56  int func(int n) {  
57      if (n == 0) {  
58          return 1;  
59      }  
60      return n * func(n - 1);  
61  }
```



RECURSION: NON-TAIL

A recursive function with pending operation(s) to be performed on return from a recursive call

```
63  int func_aux(int n, int result) {  
64      if (n == 1) {  
65          return result;  
66      }  
67      return func_aux(n - 1, n * result);  
68  }  
69  
70  int func(n) {  
71      return func_aux(n, 1);  
72  }
```



SUMMARY

- This simple iterative function mirrors the definition of factorial.
- While iterative functions have two local variables, Recursive functions have none.
- To successfully use recursion to solve a problem, we must be able to break the problem down into subproblems, that are similar in form to the original problem.



FURTHER READING RESOURCES

- Kernighan, B. W., & Ritchie, D. M. (2002). The C programming language.
- Bailey, T. (2005). An introduction to the c programming language and software design. July-2005
- CMSC 202 Lecture Notes: Recursion.
<https://redirect.cs.umbc.edu/~chang/cs202.f98/readings/recursion.html>



DEPARTMENT OF
COMPUTER SCIENCE



THANK
YOU